maximum elements present in the tree? Remember that if 'y' is in left subtree of node *x*, then key(y) is less than or equal to key(x). If 'y' is in right sub-tree of node *x*, then key(y) is greater than key (x). Since this property is satisfied at each node, the minimum element will be located at the leftmost node of the tree. The maximum element will be located at the rightmost node of the tree.

Given below is the pseudo-code for functions *FindMinimumInBST* and *FindMaximumInBST*. These functions take node 'X' which is the root of the BST.

```
FindMinimumInBST(X)
{
    while (left(X) != NULL)
        X = left(X);
    return X;
}
FindMaximumInBST(X)
{
    while (right(X) != NULL)
        X = right(X);
    return X;
}
```

What is the worst case time complexity of *FindMinimumInBST* and *FindMaximumInBST*? These functions traverse the BST from the root to the leaf. Hence their complexity is of the order of O(h) where 'h' is the height of the BST. Now the interesting question is that, given a binary search tree with N elements, what is the best possible height of the tree and what is the worst-case value of the height of the tree? For a complete binary tree with N elements, the height is O (logN) and this is the best-case value for the height of the tree. The worst-case value occurs when the tree grows linearly in one direction. The worst case value is O(N). I leave it to the reader to come up with examples for the order of tree construction such that we end up with trees of heights O(logN) and O(N).

BSTs support many dynamic set operations such as *insert*, *delete* and *find*. They can be used for implementing dictionaries and priority queues. Since we have already seen how to find the maximum and minimum, I leave it to the reader to come up with the code for 'search', 'insert' and 'delete'. For insertion and deletion, you have to ensure that the BST property holds good after the operation. All basic operations take time, proportional to the height of the tree. Since the height of the tree in the best-case is O(logN), the best-case time for these operations is O(logN). Since the height of the tree in worst-case is O(N), the worst-case time for insert, search and delete operations is O(N). Hence in the worst case, the binary search tree is no better than a simple linked list for supporting these operations. There are various data structures like AVL trees, which are balanced trees. They guarantee a worst-case time of O(logN) for 'insert', 'delete' and 'search' operations by restructuring the tree so that the height remains O(logN). We shall discuss them in next month's column.

We next look at another easy problem in the context of BSTs, that of finding the successor and predecessor of a given node. Given a node X with key value equal to key[X], the successor of X is the node Y such that key[Y] is the smallest key greater than key[X]. If node X is the maximum node in the binary search tree, then X has no successor. Note that finding the successor and predecessor requires no key comparisons. It requires only a tree traversal checking the tree structure.

If node 'X' has a non-empty right subtree, then the successor of X is the minimum node in X's right subtree. If node X has an empty right subtree, X's successor is the node Y for which X is found to be the predecessor—that is, X is the maximum in the left subtree of node Y. Given below is the pseudo-code for finding the successor of a given node X:

```
struct node FindSuccessor(struct node X)
{
    if (right[X] is != NULL)
    {
      return FindMinimumInBST(right[X]);
    }
     Y = parent[X];
     while ((Y != NULL) && (X == right[Y]))
     {
          X = Y;
          Y = Parent[Y];
     }
     return Y ;
}
```

I leave it to the reader to come up with the code for *FindPredecessor*.

This month's takeaway problem

This month's takeaway problem comes from a reader, Rajeev Kumar. This is related to the inversions that form the basis for analysing the time complexity of sorting algorithms.

Inversion of a permutation is defined as follows:

- Permutation: 6 2 3 1 4 7 8 9 5
- Inversion: 3 1 1 1 4 0 0 0 0

Here, 1 has inversion 3, as there are three elements greater than 1 which are left to 1 in the given permutation, and these are 6, 2, and 3. Similarly, 2 has inversion 1 because there is only one element left to 2 which is greater than 2, that is, 6. The problem is to find inversion of a permutation in an efficient manner. It is easy to come with an algorithm with O(n^2) time complexity, but can we find inversion of a permutation in O(nlogn) time complexity even at the cost of some memory?

If you have any favourite programming puzzles that you would like to discuss on this forum, please send them to me. Feel free to send your solutions and feedback to me at sandyasm_AT_yahoo_DOT_com. Till we meet again next month, happy programming! END

About the author:

Sandya Mannarswamy. The author is a specialist in compiler optimisation and works at Hewlett-Packard India. She has a number of publications and patents to her credit, and her areas of interest include virtualisation technologies and software development tools.